Lektion 8

Tråde og CPU-Schedulering

Computerarkitektur og Operativsystemer



Emil Lykke Diget

Syddansk Universitet

eld@mmmi.sdu.dk

Lektionsoversigt

- Lektion 1: Kombinatoriske Logiske Kredse
- Lektion 2: En Simpel Computer
- Lektion 3: Hukommelse
- Lektion 4: Mikroarkitektur
- Lektion 5: Micro-assembly og IJVM
- Lektion 6: Optimering af Mirkoarkitekturdesign
- Lektion 7: Introduktion til operativsystemer
- Lektion 8: Tråde og CPU-schedulering
- Lektion 9: Processynkronisering
- Lektion 10: Deadlocks
- Lektion 11: Memory management
- Lektion 12: Virtuel hukommelse

Outline

Multitrådet programmering

Multi-tråd-modeller

Process-schedulering

Multiprocessor-schedulering

Realtids CPU-schedulering

Multitrådet programmering

Multitrådet programmering

- Paralellisme i forskellige niveaur.
- Som vi kiggede på i sidste uge, kan man køre forskellige processer samtidigt.
- I denne uge skal vi bla. kigge på *tråde*, der muliggør paralellisme i én proces.

Definition 1 (Proces) Et instantieret program kaldes en proces. En proces har sit eget memory-adresse-område. En proces kan have en eller flere *tråde*.

Definition 2 (Tråd) En tråd "bor" i processen og deler dens memory-adresse-område og ressourcer. De har deres egne registre, program counter og stack.

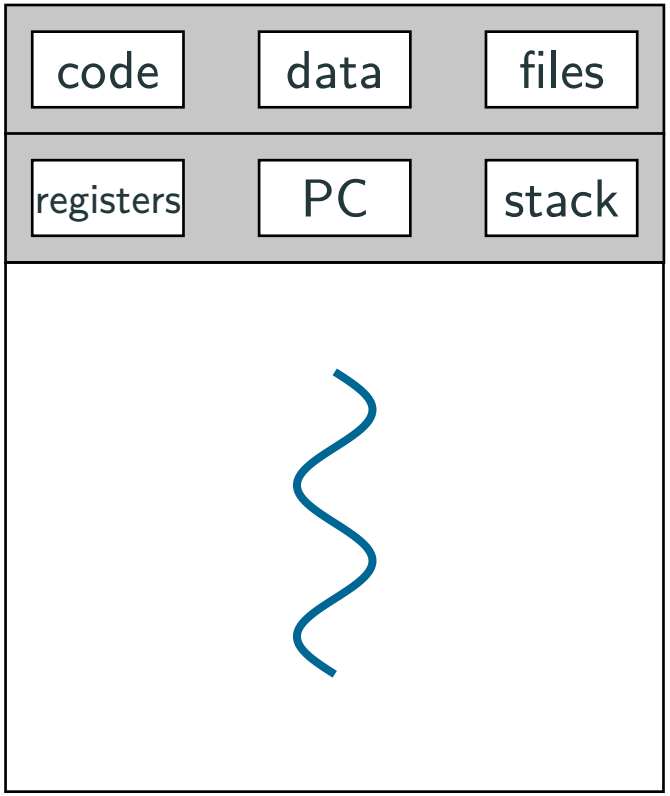


Figure 1: Single-threaded process.

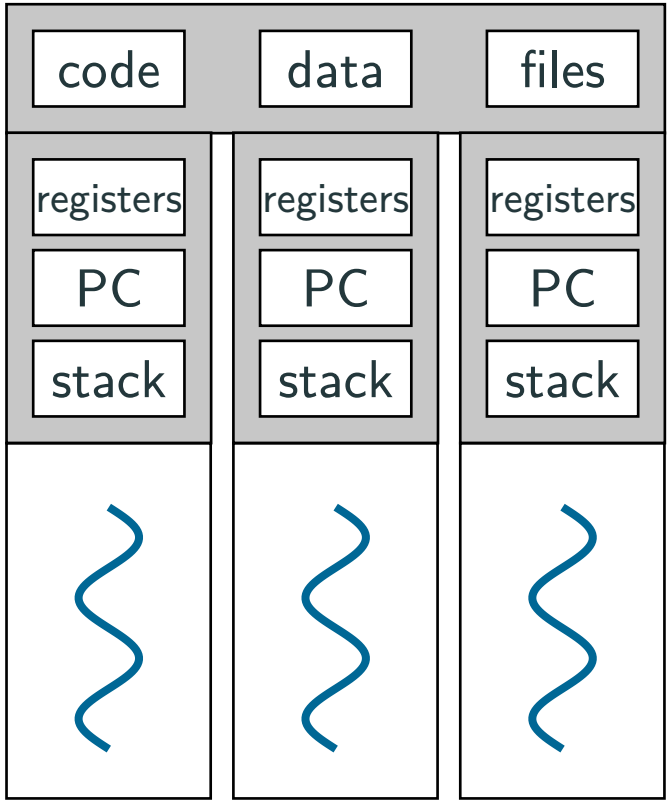
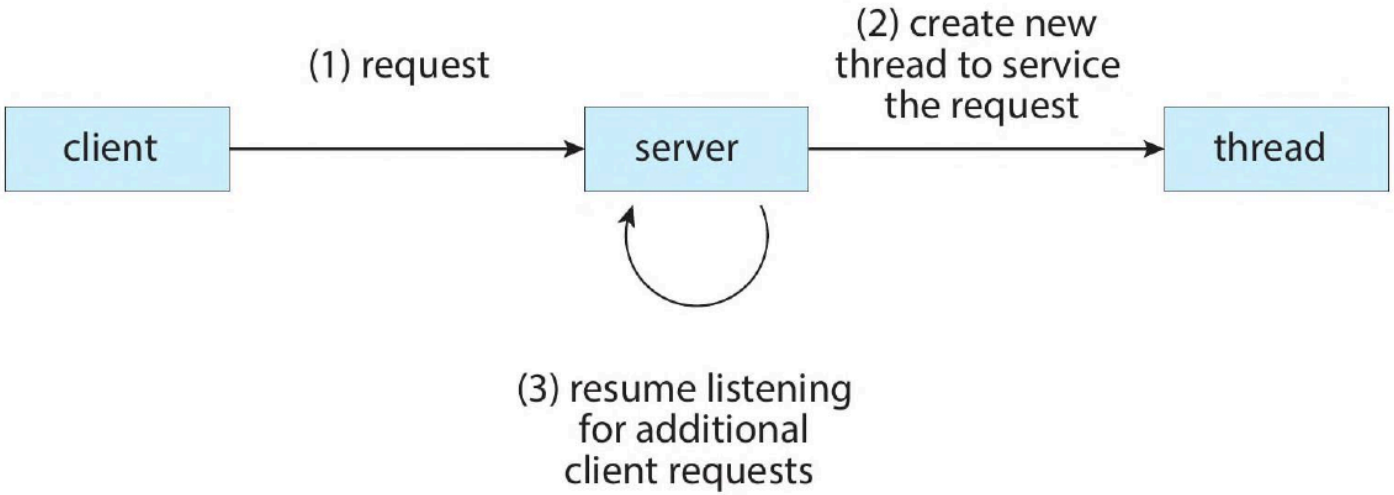


Figure 2: Multi-threaded process

Eksempel: Multi-trådet server



Responsivitet Hurtigere interaktion/reaktion.

Ressourcedeling Giver mulighed for tråde til at samarbejde om opgaver.

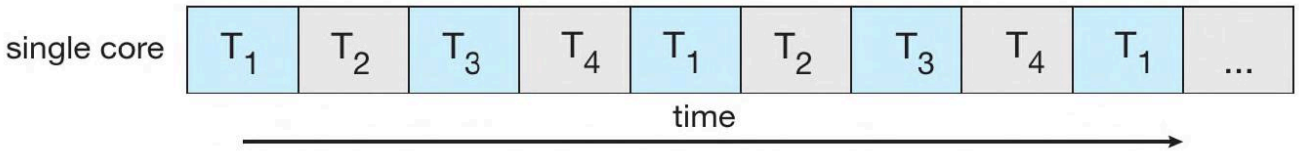
Mindre omkostningsfyldt Tråde er hurtigere at schedulere i forhold til processer, da de *lever* i samme miljø.

Skalerbarhed Tråde kan flyttes og køres på andre CPU-kerner. Kan give andre udfordringer...

Betragt et program med fire tråde: $\{T_1, T_2, T_3, T_4\}$.

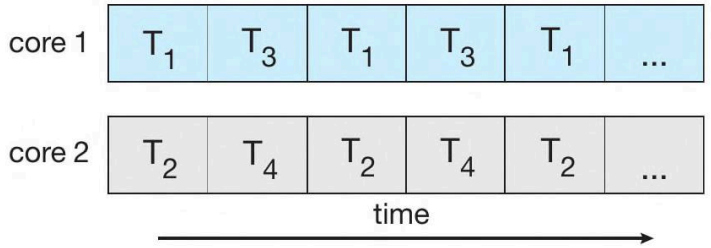
Sekventiel programafvikling

Schedulering skaber en illusion om parallelisme. Når man kun har én kerne.



Samtidig programafvikling

Ægte parallelisme. Når man har flere kerner.



Amdahls lov

Potentiel speedup ved at parallelisere koden kan regnes vha. Amdahls lov.

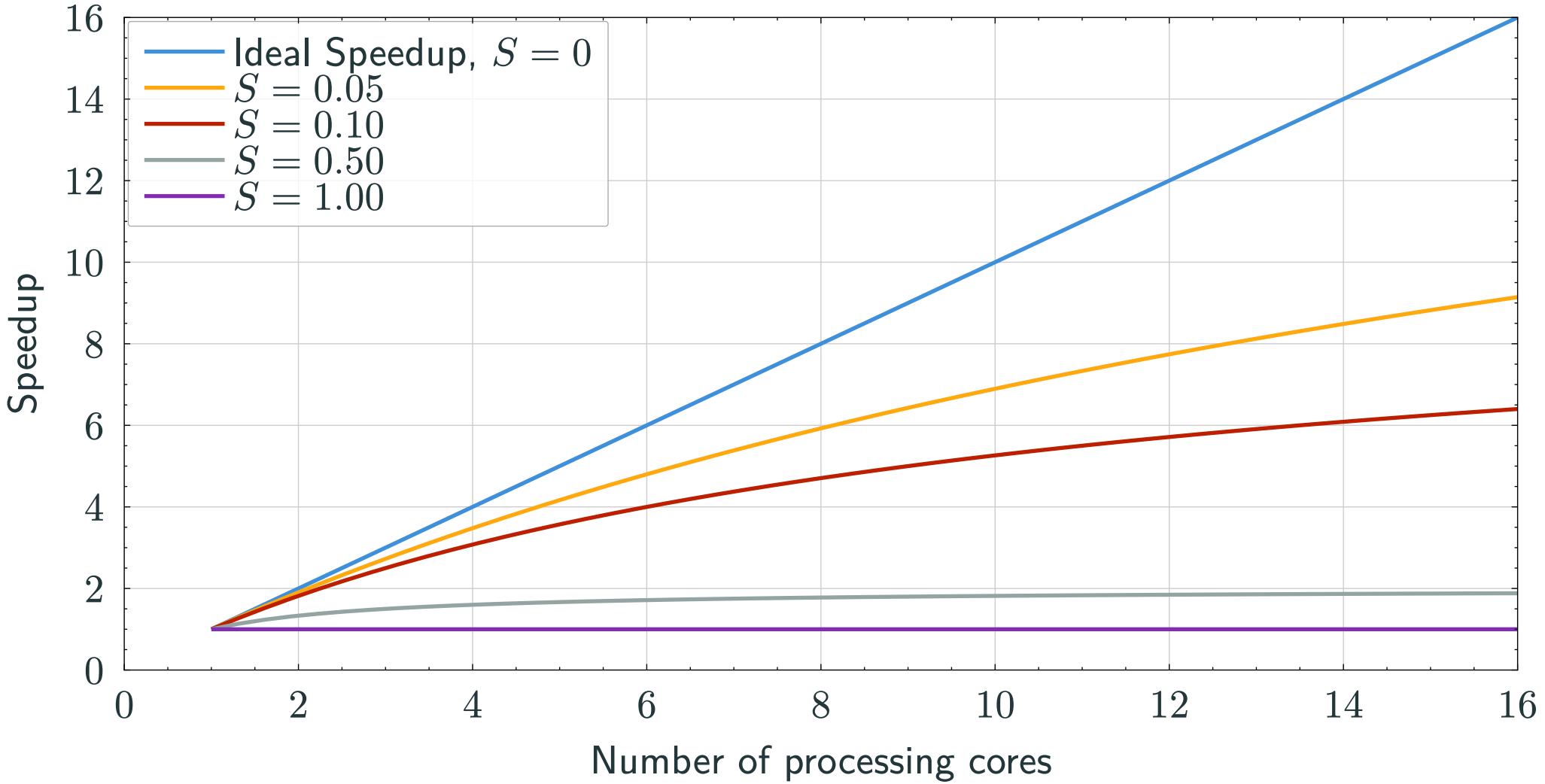
$$\text{speedup} \leq \frac{1}{S + \frac{1-S}{N}},$$

hvor
$S \in \mathbb{R}$ er den procentuelle serielle andel af en applikation,
$(1 - S) \in \mathbb{R}$ er den procentuelle parallelle andel af en applikation,
$N \in \mathbb{R}$ er antallet af CPU-kerner.

Eksempel:

- $S = 0.5,\ N = 8,\ \text{speedup} = 1.78$
- $S = 1,\ N = 8,\ \text{speedup} = 1.$ $\Longrightarrow$ **Fuldstænding seriel.**
- $S = 0,\ N = 8,\ \text{speedup} = 8.$ $\Longrightarrow$ **Fuldstænding parallel.**

Amdahls lov


Ideal Speedup, $S = 0$
$S = 0.05$
$S = 0.10$
$S = 0.50$
$S = 1.00$
Speedup
Number of processing cores
0
2
4
6
8
10
12
14
16

Udfordringer ved multicore-programmer

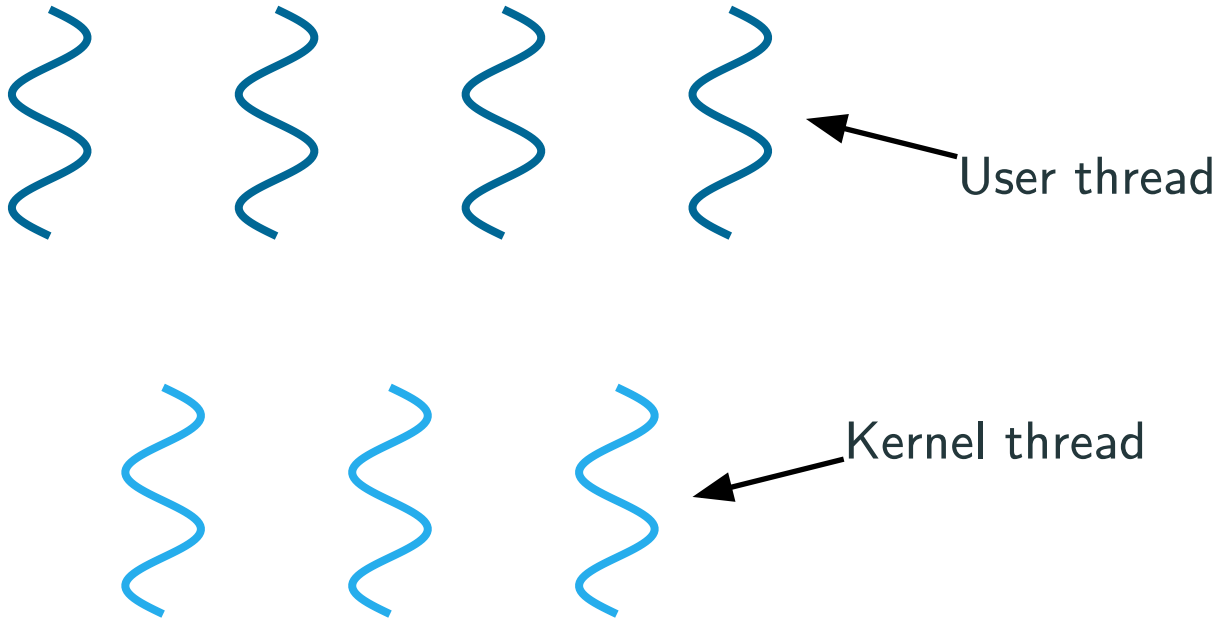
Det kan være udfordrende at lave et parallelt program:

1. Identifikation af opgaver egnet til hver sin kerne.
2. Balancering af belasting.
3. Opdeling af data.
4. Synkronisering af data mellem tråde, hvis dette tilgås af flere samtidigt.
5. Efter at programmet er skrevet færdigt, starter et omfattende arbejde i at teste og debugge, da flere tråde giver flere eksekveringsstier.

Multi-tråd-modeller

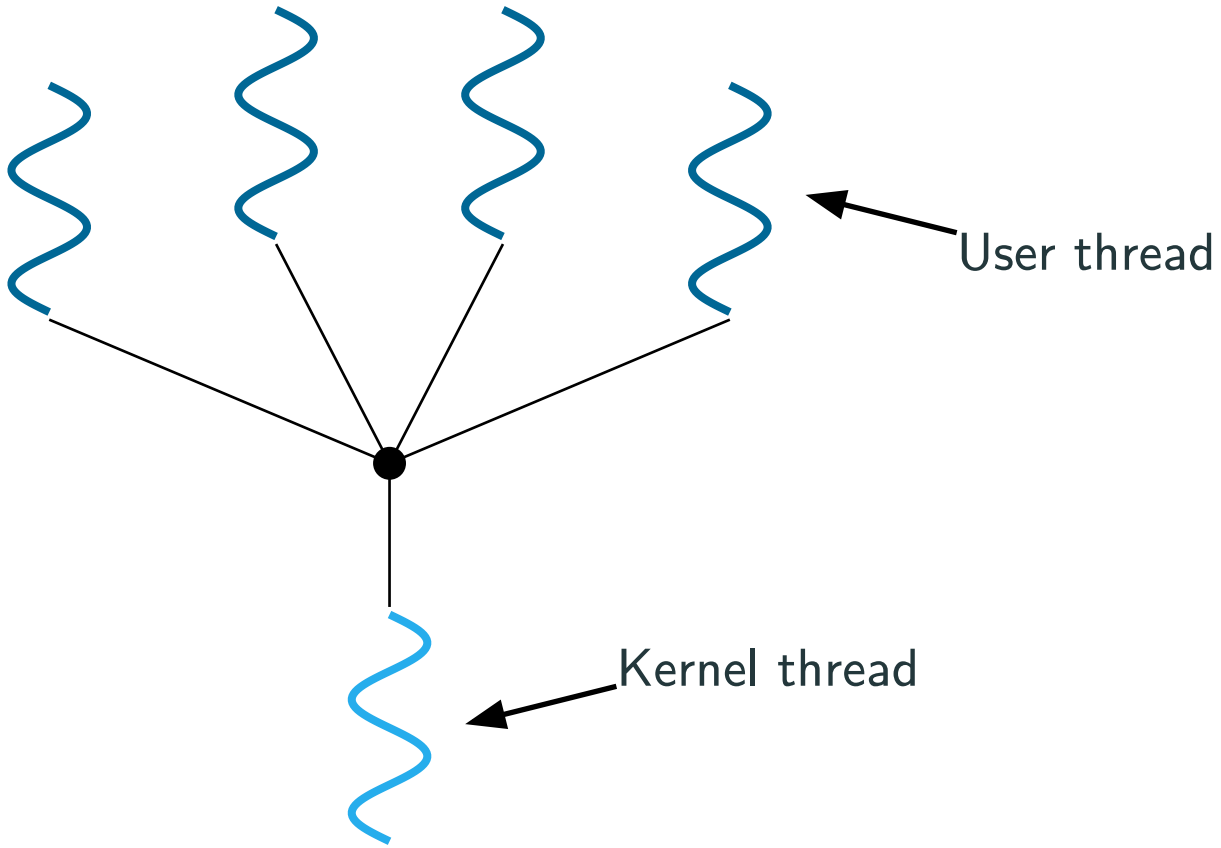
Multi-tråd-modeller

- Tråde eksisterer både i user- og kernel space.
- Kernel threads er styret direkte af operativsystemet.
- Her skal vi kigge på, hvordan user- og kernel-tråde kan være forbundet.



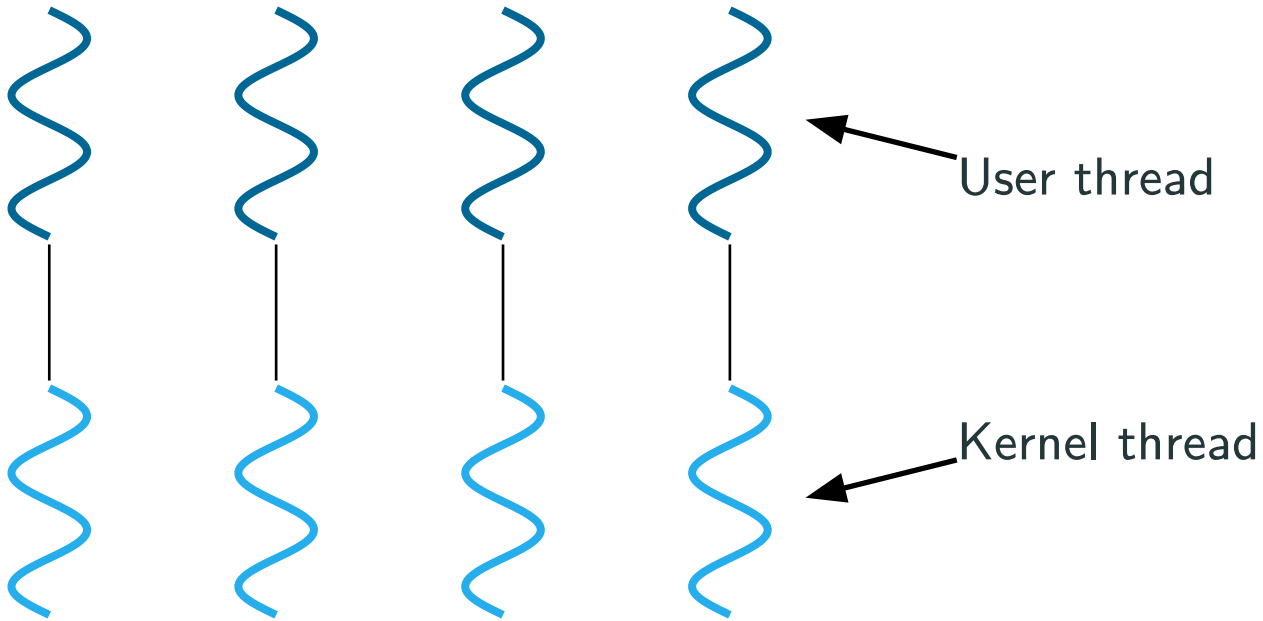
Mange-til-én (many-to-one)

- Mange bruger-tråde mappet til én kernel-tråd.
- *Alt* vil blokere, hvis en bruger-tråd laver et blokerende system-kald.



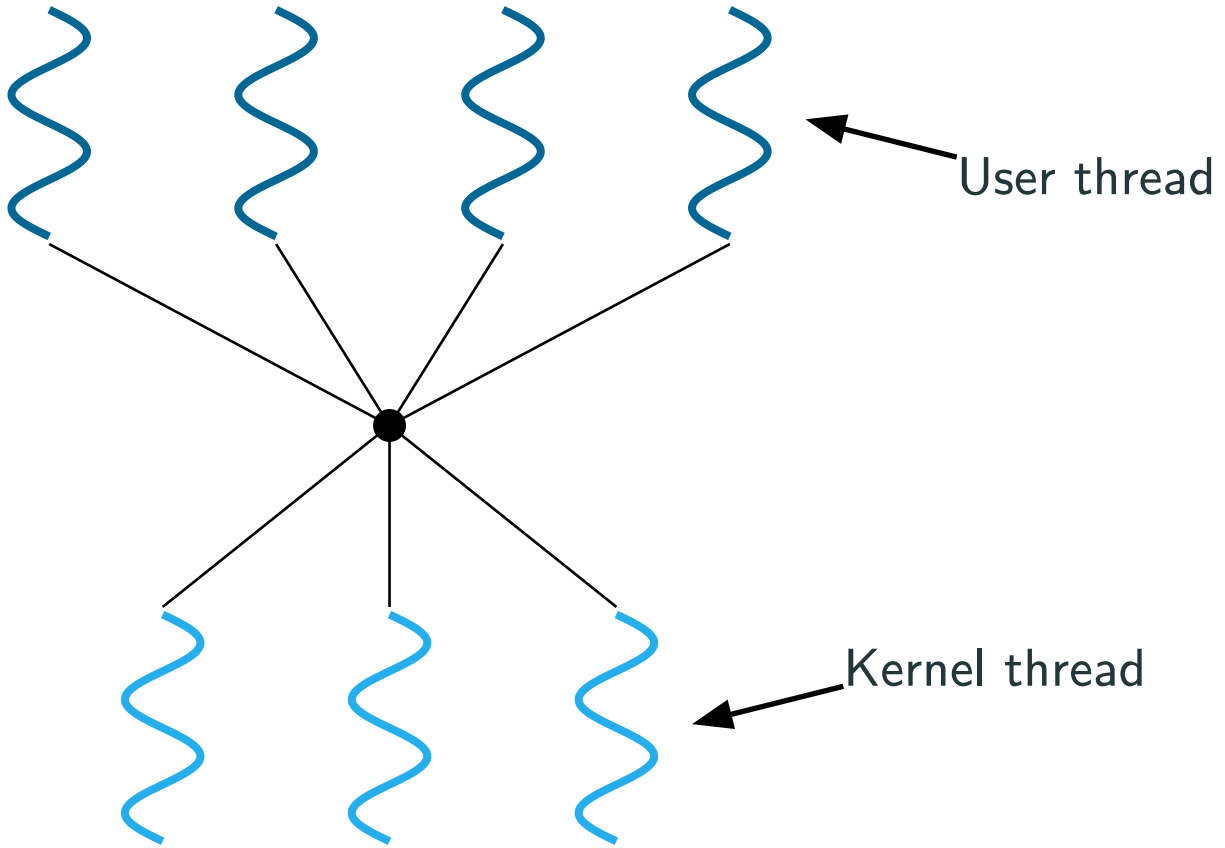
Én-til-én (one-to-one)

- Allokering af én kerne-tråd til hver bruger-tråd.
- Giver mulighed for parallel eksekvering.



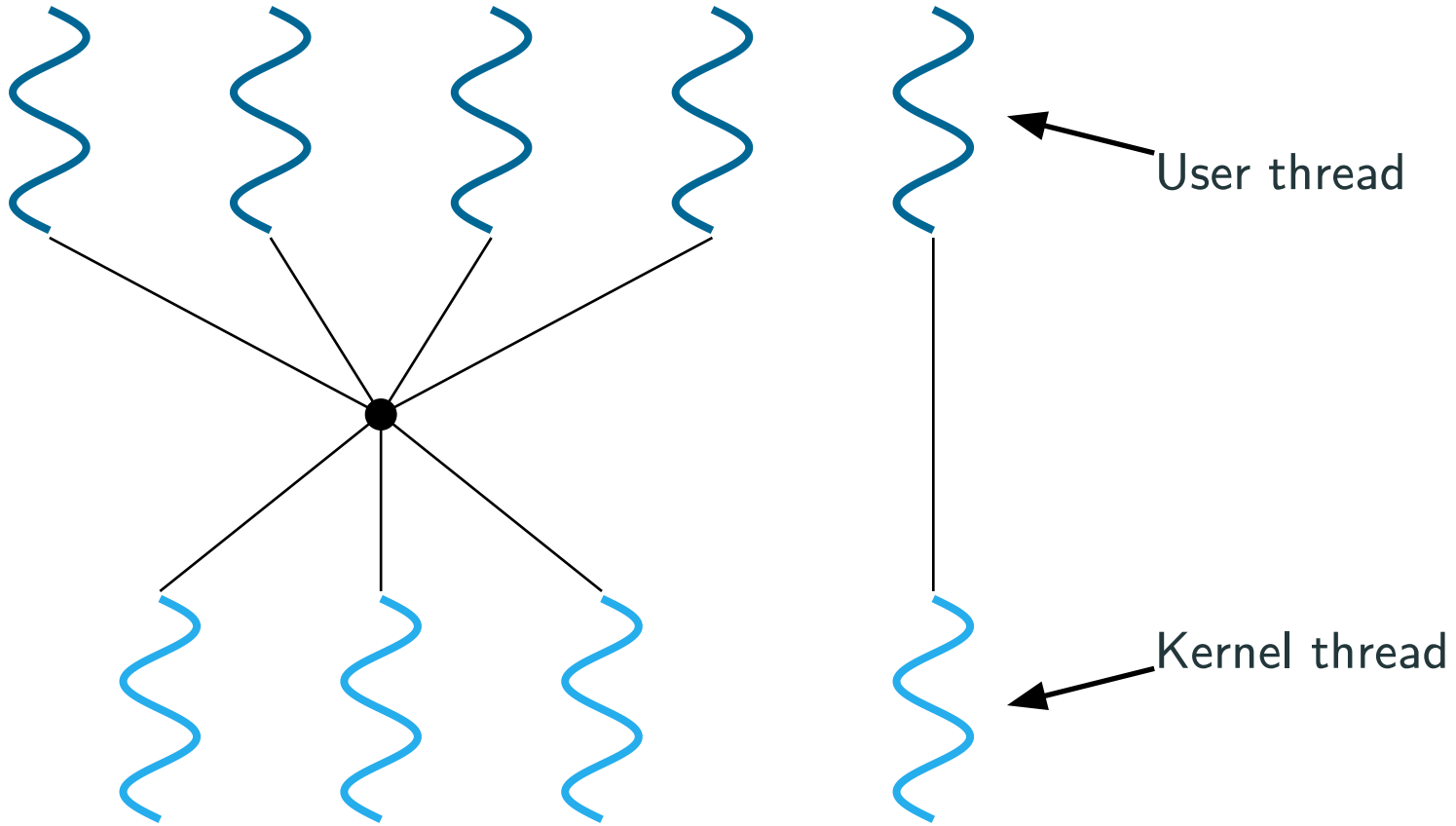
Mange-til-mange (many-to-many)

- Fleksibel model.
- Ingen øvre grænse på antallet af bruger-tråde.
- Kerne-tråde kan køre parallelt.
- Udfordrende at implementere i praksis.



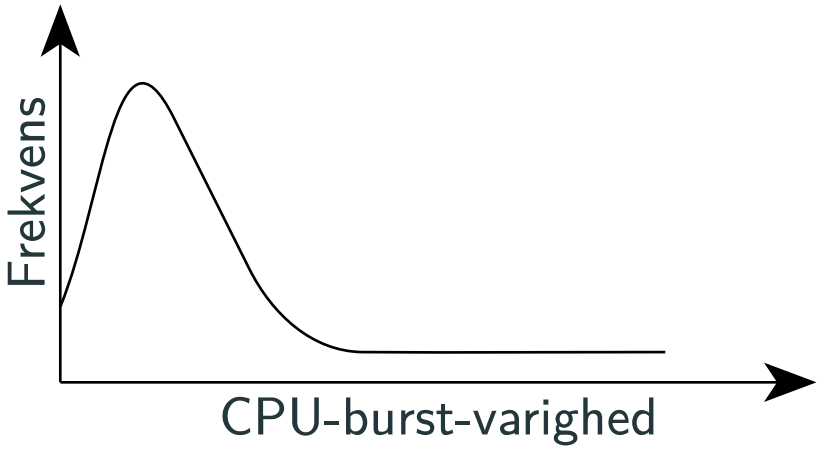
Two-niveau (two-level)

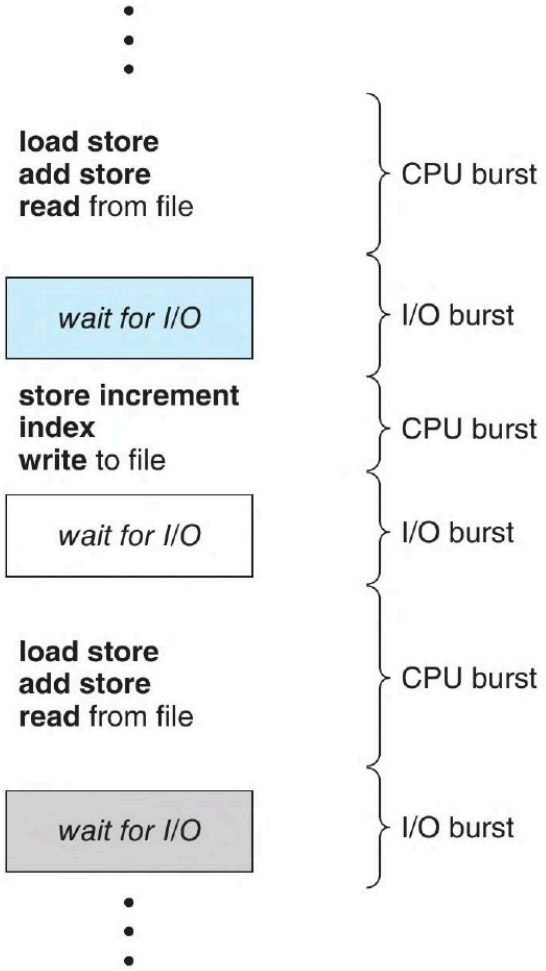
- Ligner mange-til-mange.
- En bruger-tråd kan knyttes direkte til en kernetråd.



Process-schedulering

- En process skifter mellem to states: **CPU burst** og **I/O burst**





Altså der er flest korte CPU-bursts.

Design-mål

For at sammenligne scheduleringsalgoritmer, er der forskellige mål, der kan overvejes:

CPU-udnyttelse CPUen skal udnyttes mest muligt.

Throughput Hvor mange brugerprocesser, der gennemføres per tidsenhed.

Turnaround time Tid fra en brugerprocess får ressourcer til den frigiver dem igen.
Summen af ventetid i alle køer og CPU-tid.

Ventetid Hvor lang tid processen venter i ready-køen i dens levetid.

Responstid Tid fra en request registreres til et respons startes.

Desuden

Varians i responstiden En lille varians i responstiden ønskes, da en stor varians kan give indtryk af et ustabilt system.

First-Come, First-Served (FCFS)

- Den simpleste metode.
- Implementeres nemt ved en FIFO-kø.

Process	Burst-tid
P_1	24
P_2	3
P_3	3



Ventetid:

$$T_1 = 0, \quad T_2 = 24, \quad T_3 = 27 \quad \Longrightarrow \quad \overline{T} = \frac{0 + 24 + 27}{3} = 17$$

First-Come, First-Served (FCFS)

Processernes rækkefølge har betydning...



Ventetid:

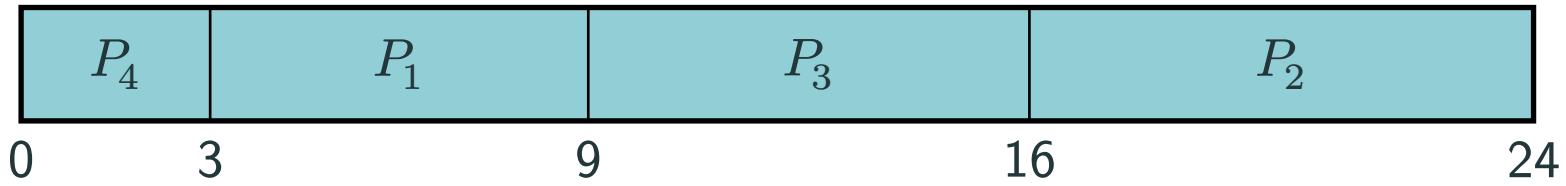
$$T_1 = 6, \quad T_2 = 0, \quad T_3 = 3 \quad \Longrightarrow \quad \overline{T} = \frac{6+0+3}{3} = 3$$

Shortest-Job-First (SJF)

Non-preemtiv schedulering, hvor den korteste process eksekveres først.

Det antages derved at alle processer kommer samtidigt.

Process	Burst-tid
P_1	6
P_2	8
P_3	7
P_4	3



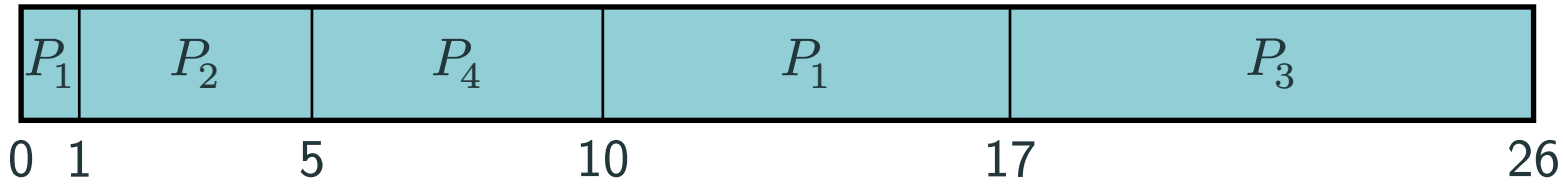
Ventetid:

$$T_1 = 3, \quad T_2 = 16, \quad T_3 = 9, \quad T_4 = 0 \quad \Longrightarrow \quad \overline{T} = \frac{3 + 16 + 9 + 0}{4} = 7$$

Shortest-Job-First (SJF)

Preemtiv schedulering, hvor den korteste process, der er ankommet, eksekveres først.

Process	Ankomsttid	Burst-tid
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5



Ventetid:

$$T_1 = 10 - 0, \quad T_2 = 1 - 1, \quad T_3 = 17 - 2, \quad T_4 = 5 - 3$$

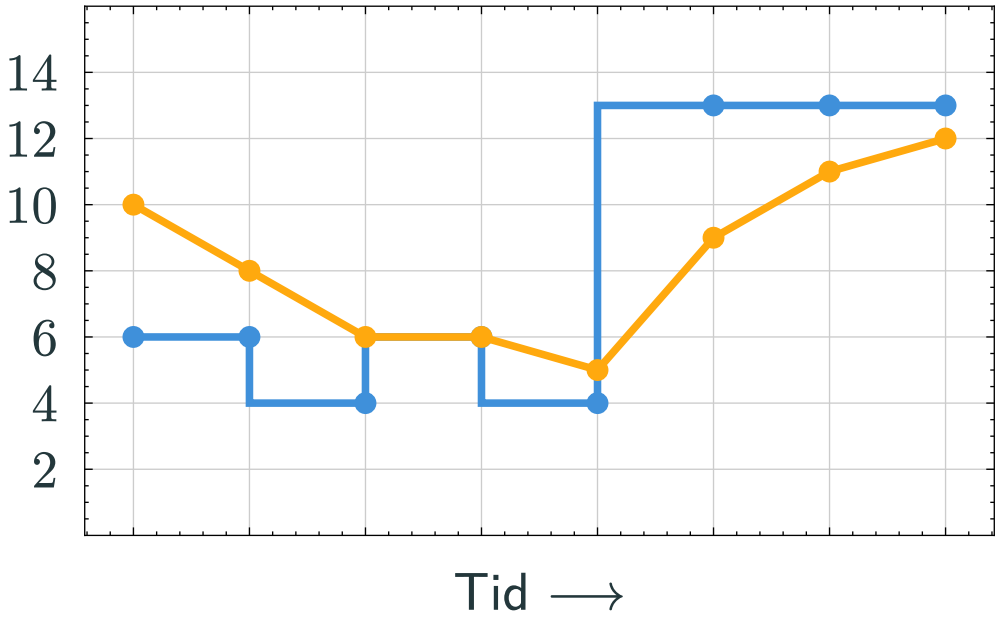
$$\Longrightarrow \quad \overline{T} = \frac{(10-0) + (1-1) + (17-2) + (5-3)}{4} = 6.75$$

Estimering af CPU-burst

Længden af den næste CPU-burst kan ikke kendes på forhånd, så den estimeres typisk vha. et *exponential average* filter:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n,$$

hvor $0 \leq \alpha \leq 1$ styrer vægtningen af nye kontra gamle målinger.



CPU-burst: [6, 4, 6, 4, 13, 13, 13]

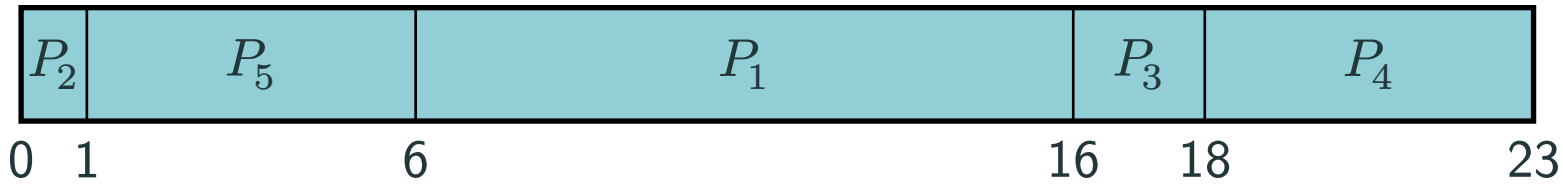
Filtered: [10, 8, 6, 6, 5, 9, 11, 12], med $\tau_0 = 10$, $\alpha = 0.5$.

Prioritets-scheduling

Prioritet angiver vigtigheden af en process.

Her kommer alle processerne samtidigt, og scheduleringen er ikke-preemptiv.

Process	Burst-tid	Prioritet
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

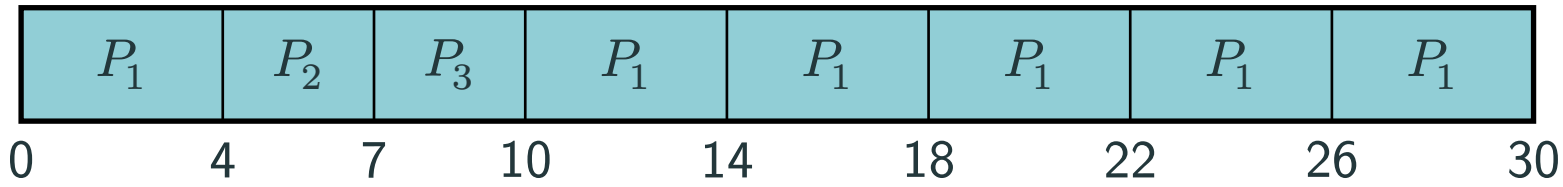


Bemærk, der kan opstå starvation, hvis der f.eks. kommer nye processer ind med højere prioritet.

Round-Robin (RR)

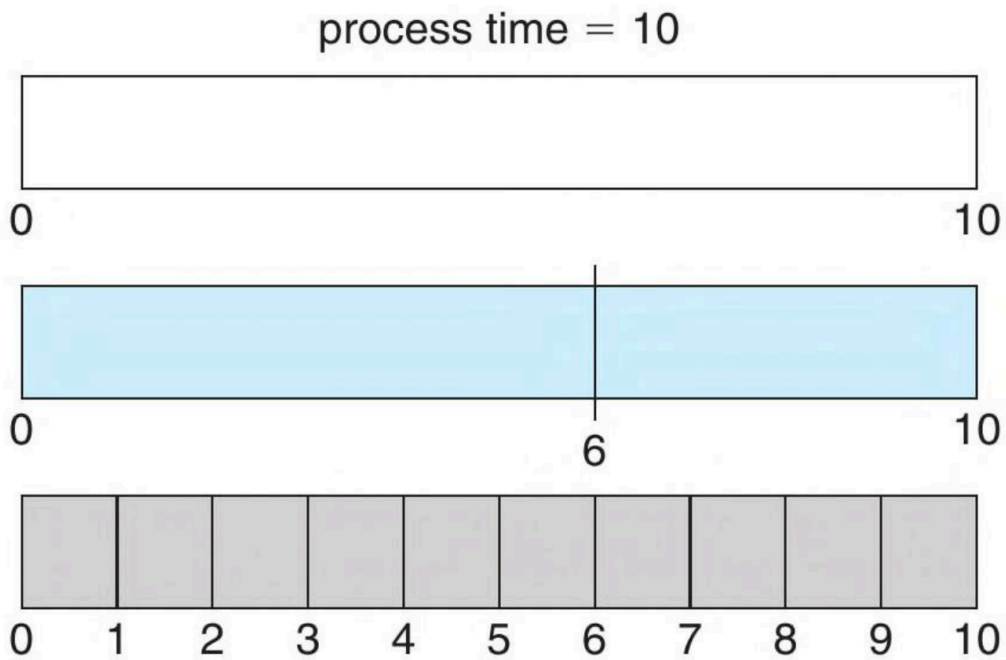
Tiden deles op i bidder, såkaldte *time slices* eller *kvantum*, her er det 4.

Process	Burst-tid
P_1	24
P_2	3
P_3	3



Hvis *time slice* $\rightarrow \infty$, bliver algoritmen til *First Come First Served* (FCFS).

Kontekst-skift bidgrager med administration

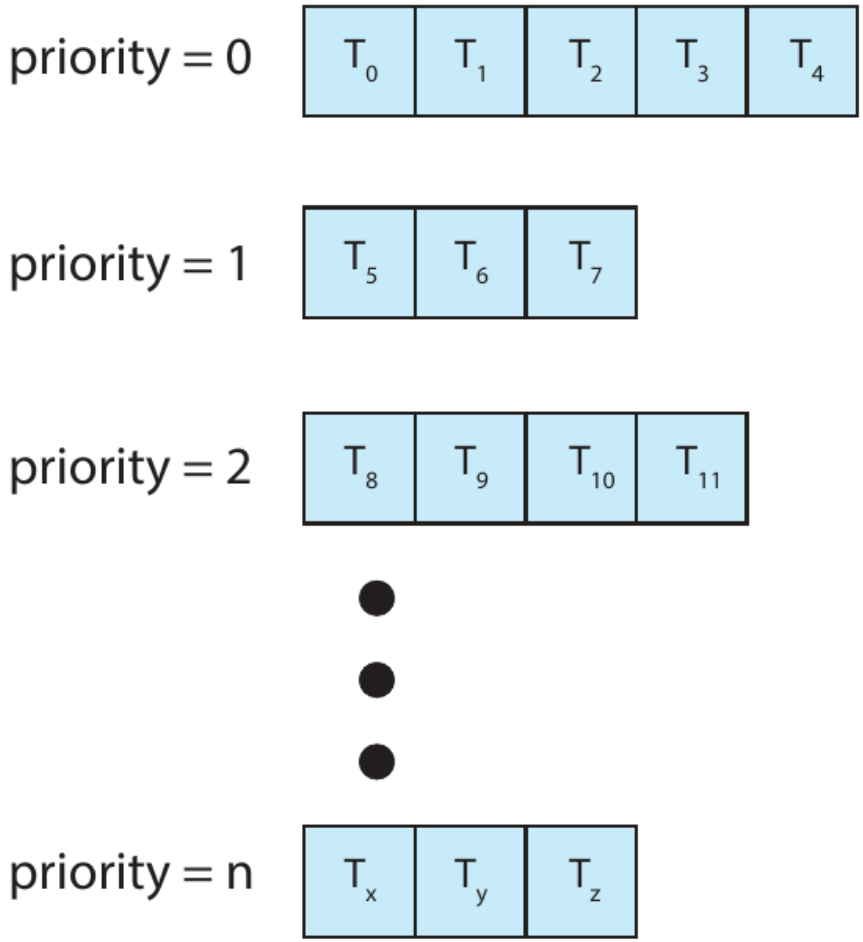


quantum	context switches
12	0
6	1
1	9

$\Longrightarrow$ Et kortere tids-kvantum giver mere process-administration.

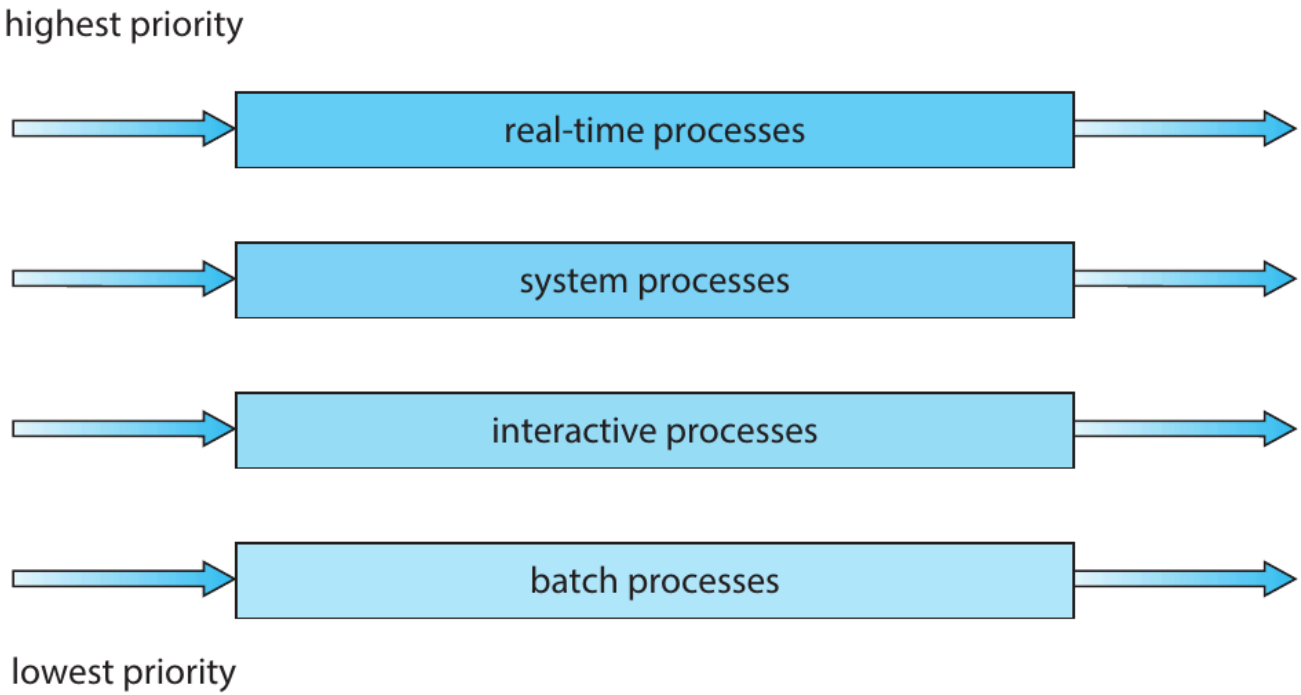
Multilevel Queue Schedulering

- Seperat kø for hver prioritet.
- Hver kø har deres egen scheduler.
- Går kun ned i prioritet, når kø er tom.



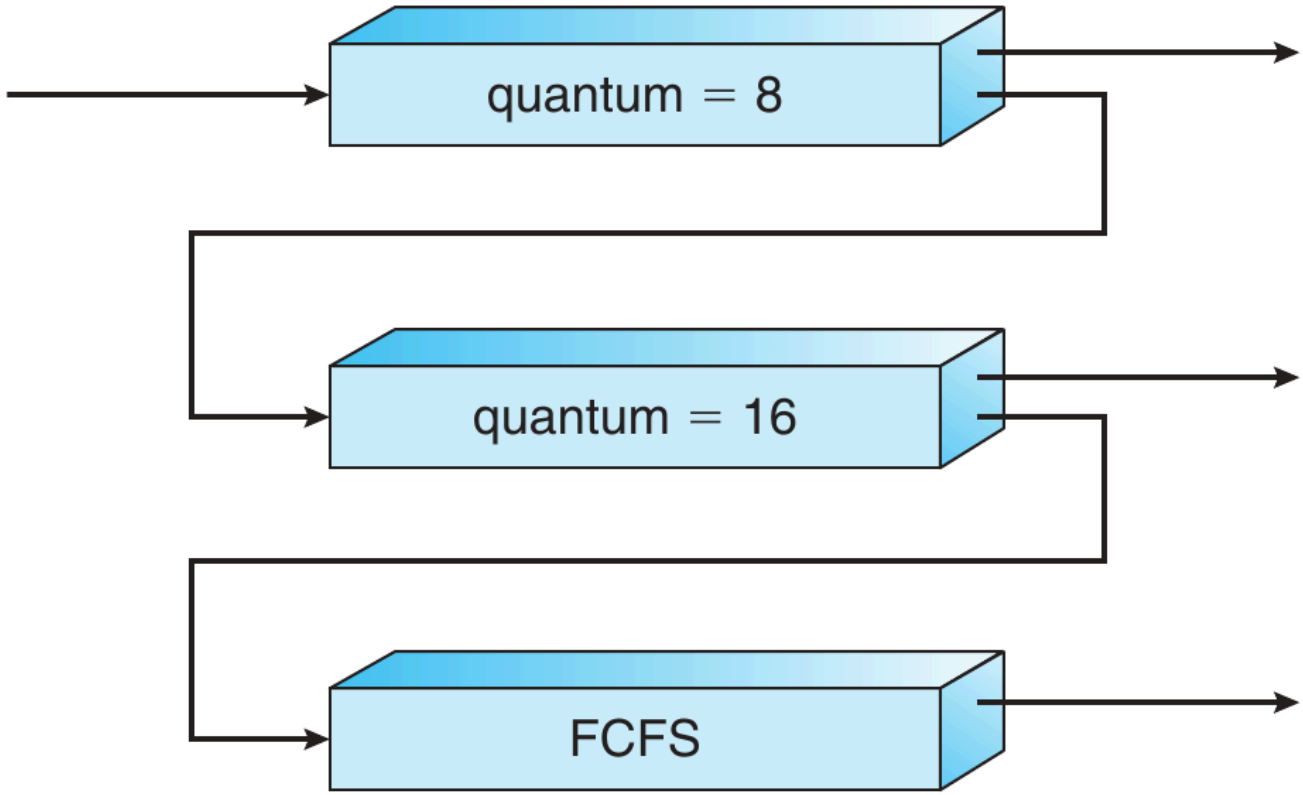
Multilevel Queue Schedulering

- Prioritet kan sættes efter processtype.



Multilevel Feedback Queue

- Alternativ: Processer kan bevæge sig mellem køer.
- Hvis en process tager længere tid end det allokerede kvantum, bevæger den sig nedad.



Multiprocessor-schedulering

Multiprocessor-scheduling

CPU-scheduling bliver mere komplekst, når der er flere CPUer i systemet.

De fleste computere, smartphones, etc., har flere kerner (logiske CPUer), så dette er i høj grad relevant.

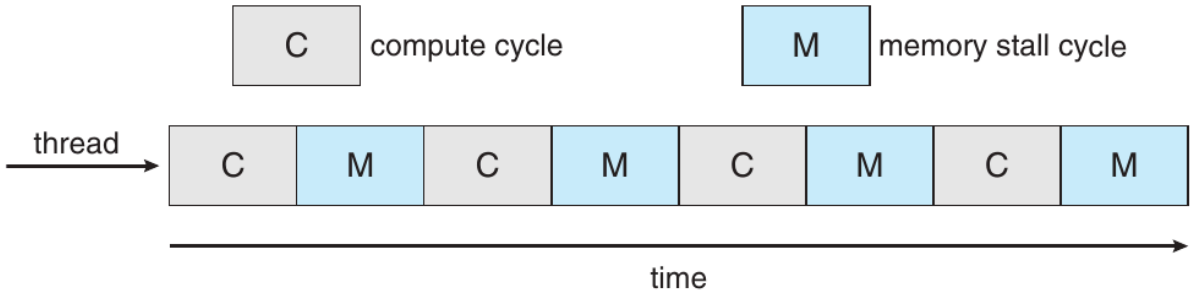
To slags:

Assymmetrisk En *master* CPU, der står for schedulering, IO-processering, etc.

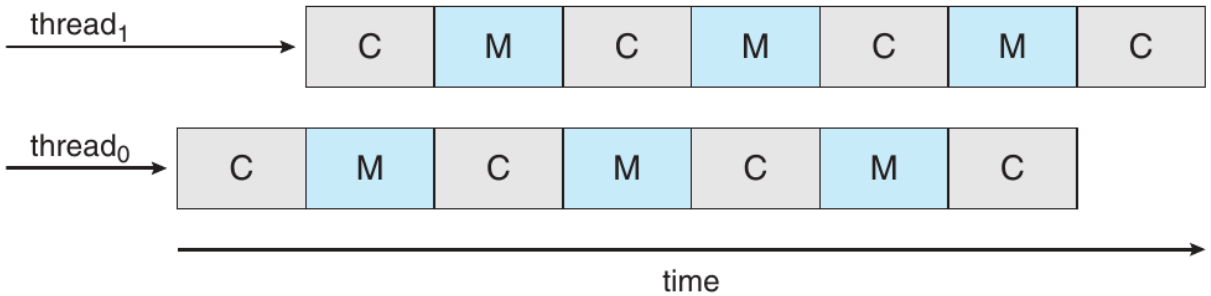
Master er administrator, og andre CPUer står for selve arbejdet.

Symmetrisk (SMP) Hver process schedulerer selv, enten med egen eller fælles kø.

Når en CPU tilgår hukommelsen skal den ofte vente på at dataen bliver tilgængelig; kaldes for *memory stall*.

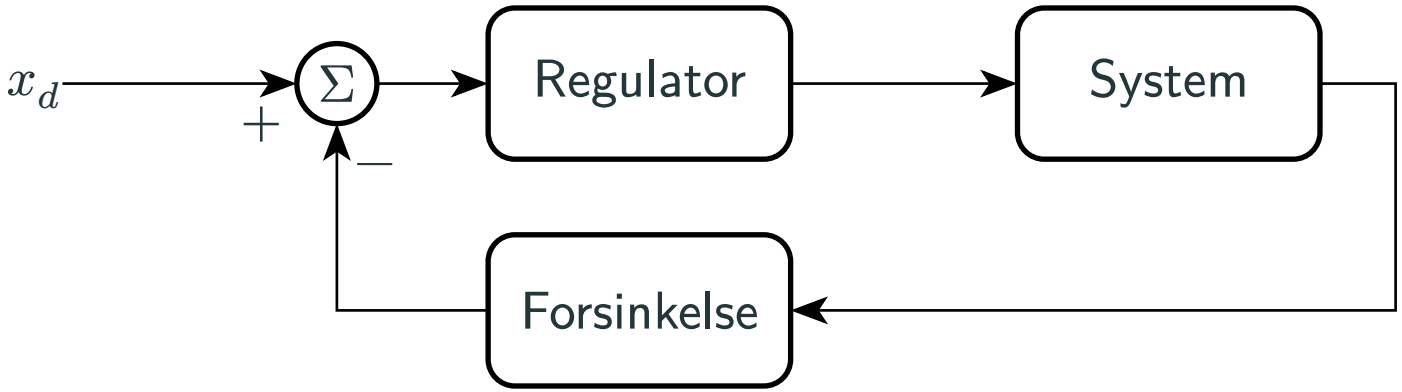


Kan løses ved at hver kerne kan have to (eller flere) hardware-tråde.



Realtids CPU-schedulering

Realtids-scheduling er **meget** vigtigt, når man f.eks. arbejder med kontrolsystemer til robotter.



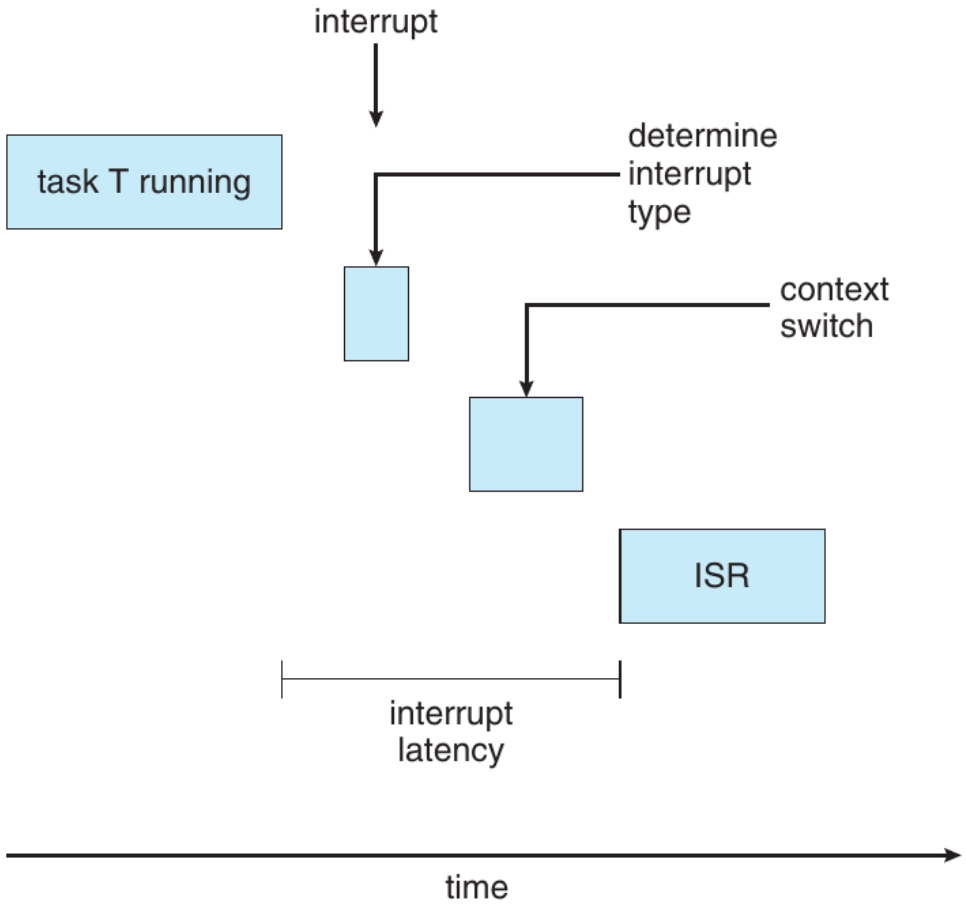
Hvis man ikke kan regne med størrelsen af forsinkelsen, så kan man ikke garantere ydeevne (performance).

- I værste fald kan man få et ustabilt system.

Realtid betyder ikke nødvendigvis "så hurtigt som muligt", men "så præcist som muligt".

Deterministisk

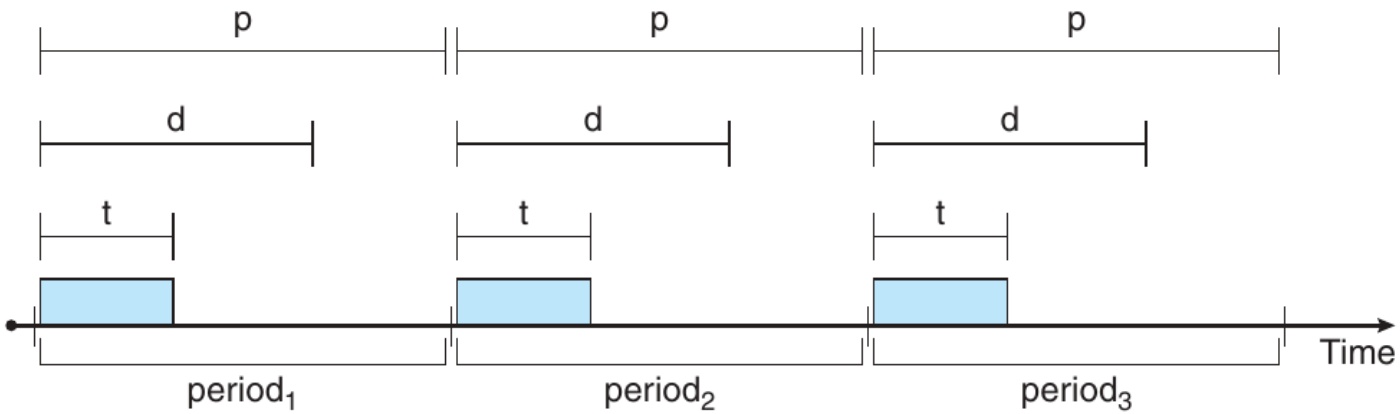
– Altså at man kan *regne* med størrelsen af forsinkelsen.



Prioritetsbaseret Realtids-scheduling

Prioritetsbaseret preeimtiv schedulering garanterer kun *soft* real-time.

- Realtidsprocessen bliver skiftet ind, men der er ingen garanti for, at den er færdig inden dens deadline.

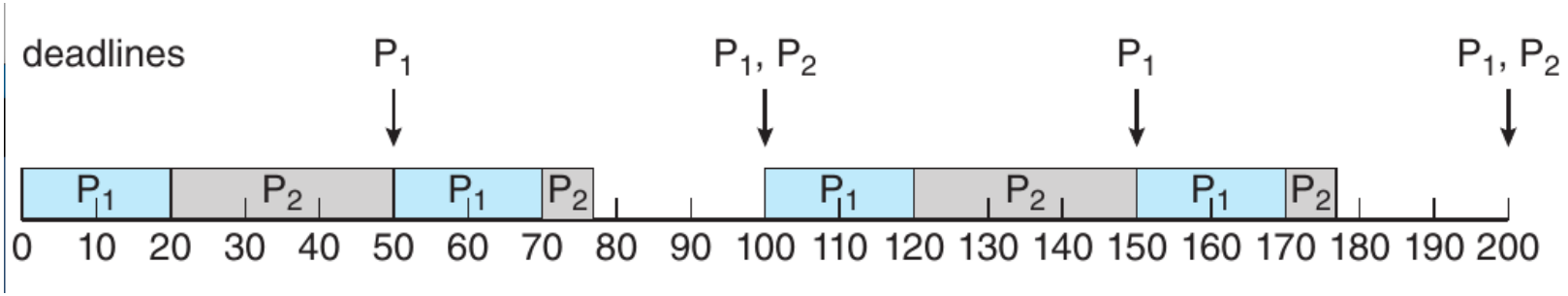


hvor p er fast periodetid, d er deadline, t er en (typisk) fast afviklingstid. Der gælder:

$$0 \leq t \leq d \leq p$$

Rate Monotonic

Hard-real time kræver at en proces når dens deadline.

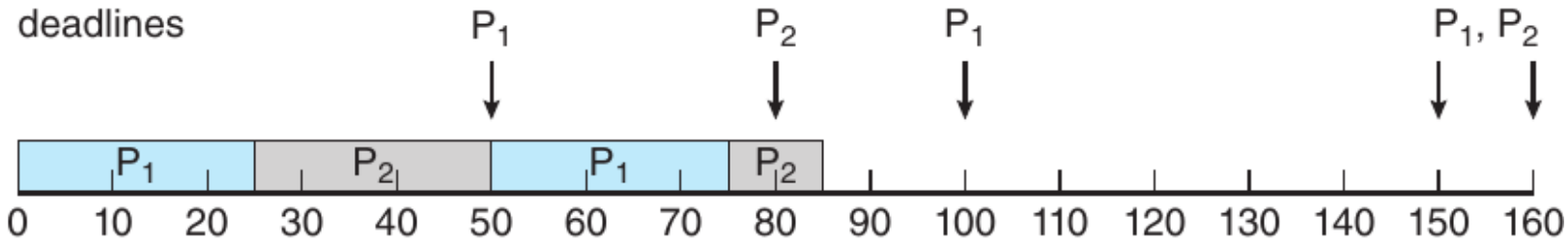


P_1 har periode 50, P_2 har periode 100.

Kortere periode $\rightarrow$ højere prioritet.

$$\text{CPU-udnyttelse} = \frac{t_1}{p_1} + \frac{t_2}{p_2} = \frac{20}{50} + \frac{35}{100} = 0.75$$

Rate Monotonic



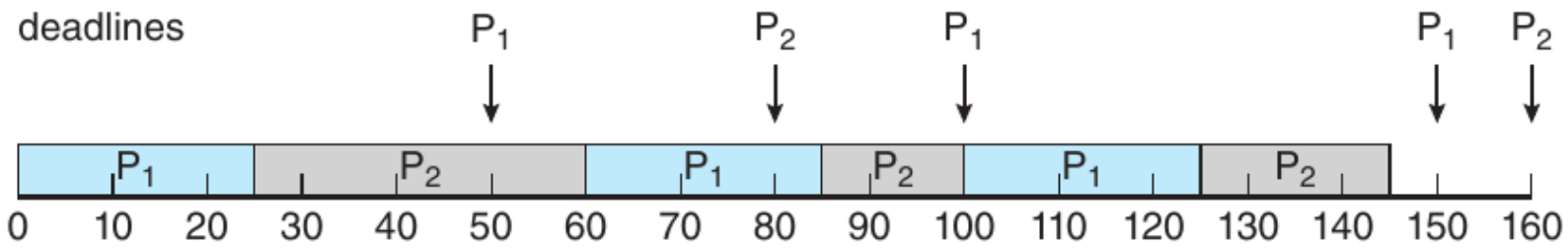
Hvis nu P_1 tager lidt længere tid end før, $t_1 = 25$, og P_2 har en lavere periode p_2, kan deadlinen ikke nås.

$$\text{CPU-udnyttelse} = \frac{t_1}{p_1} + \frac{t_2}{p_2} = \frac{25}{50} + \frac{35}{80} = 0.9375$$

CPU-udnyttelse har en øvre grænse for N processer på

$$N\left(2^{\frac{1}{N}} - 1\right) \Rightarrow \quad \sim 0.83$$

Earliest Deadline First (EDF)



Prioritet tildeles *dynamisk*. Den førstkommende deadline giver højest prioritet.

P_1 afbryder *ikke* ved 50, da P_2 her har højest prioritet.

P_1 afbryder ved 100, da den her har højest prioritet.